

Unifying reward-tilted posterior guidance with D-CBG

What proposal does D-CBG induce over $\mathbf{x}_0$?

2026-08-01

Abstract

D-CBG and our reward-tilted posterior guidance are usually described as competing classifier-based mechanisms, but they act at different levels of the generative process: D-CBG tilts the one-step posterior over $\mathbf{x}_{t-1}$, while our method tilts the denoiser’s distribution over the clean sequence $\mathbf{x}_0$. This note reads off exactly what D-CBG computes from the implementation, lifts its classifier factor to an $\mathbf{x}_0$ -level proposal, and shows that the resulting importance weights collapse to a single clean expression in which the classifier’s normalizer cancels. The combined estimator uses the classifier to decide *where the N atoms land* and the exact reward to decide *what they are worth*, and remains consistent for the reward-tilted target regardless of how bad the classifier is.

1 Notation

Let L be the sequence length, V the vocabulary, c the target class (in our experiments, “QED above the 90th percentile of QM9”), and $p_\phi(c \mid \mathbf{x}, t)$ the *noisy* classifier D-CBG trains on diffusion latents. Write $\mathbf{x}_t^{l \leftarrow k}$ for the sequence $\mathbf{x}_t$ with position l overwritten by token k . Throughout, define the **substitution score**

$$s_l(k) := \log p_\phi\left(c \mid \mathbf{x}_t^{l \leftarrow k}, t\right) \in (-\infty, 0]. \quad (1)$$

This is exactly the tensor called `classifier_log_prob` in `diffusion.py`, of shape (B, L, V) .

2 What D-CBG actually computes

Reading `Diffusion._cbg_denoise` (`diffusion.py:1344--1456`), D-CBG performs a *per-position* tilt of the unguided reverse posterior:

$$q_{\text{CBG}}\left(\mathbf{x}_{t-1}^l = k \mid \mathbf{x}_t\right) \propto q\left(\mathbf{x}_{t-1}^l = k \mid \mathbf{x}_t\right) \exp(\gamma s_l(k)) \quad (2)$$

implemented as `guided_log_probs = gamma * classifier_log_prob + diffusion_log_probs` followed by a softmax over the vocabulary axis, with the positions then sampled independently. For absorbing-state diffusion the unmasked positions are forced to copy $\mathbf{x}_t$, so (2) is only active on masked positions.

The two `use_approx` branches differ only in how $s_l(k)$ is obtained:

- **Exact** (`use_approx=False`): materialize all $L \cdot V$ single-token substitutions $\mathbf{x}_t^{l \leftarrow k}$ and run the classifier on each. Cost $L \cdot V$ classifier forwards per step — this is the variant that ran out of memory at batch size 64.

- **Approximate** (`use_approx=True`): a first-order Taylor expansion in the one-hot input,

$$s_l(k) \approx \log p_\phi(c \mid \mathbf{x}_t, t) + \left. \frac{\partial \log p_\phi(c \mid \mathbf{x}, t)}{\partial x_{l,k}} \right|_{\mathbf{x}=\mathbf{x}_t} - \left. \frac{\partial \log p_\phi(c \mid \mathbf{x}, t)}{\partial x_{l,\mathbf{x}_t^l}} \right|_{\mathbf{x}=\mathbf{x}_t}, \quad (3)$$

the last term centering the score at the token currently occupying position l . Cost: one forward and one backward per step.

Three properties worth naming. First, D-CBG’s tilt lives at the level of $\mathbf{x}_{t-1}$, not $\mathbf{x}_0$. Second, the tilt is *factorized*: it is a product of per-position factors, so it can never express a preference that couples positions. Third, D-CBG applies the tilt and stops — there is no correction step, so the sampler targets (2) rather than any exactly-specified distribution over $\mathbf{x}_0$. The approximation error of p_ϕ (and of the Taylor expansion) propagates straight into the samples.

3 What our method computes

Our estimator tilts at the level of the clean sequence. With $\mathbf{x}_0^{(n)} \sim p_\theta(\mathbf{x}_0 \mid \mathbf{x}_t)$ drawn i.i.d. from the denoiser,

$$w_n = \frac{\exp(\lambda r(\mathbf{x}_0^{(n)}))}{\sum_m \exp(\lambda r(\mathbf{x}_0^{(m)}))}, \quad \bar{\mathbf{x}}_0 = \sum_{n=1}^N w_n \text{onehot}(\mathbf{x}_0^{(n)}), \quad (4)$$

and $\mathbf{x}_{t-1}$ is drawn from the posterior with $\bar{\mathbf{x}}_0$ substituted for $\mathbf{x}_0$. This is self-normalized importance sampling for the target $q(\mathbf{x}_0) \propto p_\theta(\mathbf{x}_0) \exp(\lambda r(\mathbf{x}_0))$, so unlike (2) it is *consistent*: as $N \rightarrow \infty$ it recovers the exact reward-tilted distribution. The reward r sees clean $\mathbf{x}_0$, so it may couple positions arbitrarily — QED is a whole-molecule functional — and needs no training.

The price is the atomic support discussed in the companion note: only the N sampled candidates carry mass, and they are sampled from the *untitled* denoiser, so the atoms land wherever the base model happens to look rather than where the reward is high.

4 The unification

The two mechanisms are complementary precisely because of the above: D-CBG has a cheap factorized signal that can move probability mass but cannot be corrected, and we have an exact uncorrectable-by-nature signal that can only reweight mass that is already there. The combination is to use the first as the **proposal** and the second as the **weight**.

4.1 What the optimal factorized proposal is

The obstacle is that (2) tilts $\mathbf{x}_{t-1}$, whereas importance sampling needs a distribution over $\mathbf{x}_0$. Before borrowing anything from D-CBG, it is worth deriving what we *should* be looking for, because that derivation is what tells us the classifier is the right object rather than a convenient one.

Two facts frame the search.

(i) Any factorized proposal is legal. Self-normalized importance sampling is consistent for the target $q(\mathbf{x}_0 \mid \mathbf{x}_t) \propto p_\theta(\mathbf{x}_0 \mid \mathbf{x}_t) e^{\lambda r(\mathbf{x}_0)}$ for *any* proposal $\tilde{p}$ with $\text{supp}(\tilde{p}) \supseteq \text{supp}(q)$, provided the weights carry the ratio $q/\tilde{p}$. So the choice of proposal is not a correctness question at all — it cannot make the estimator wrong, only slow. What it controls is the effective sample size, i.e. how many of the N atoms actually count. We restrict to *factorized* $\tilde{p}$ purely for tractability: a factorized distribution is the only kind we can draw N i.i.d. samples from at the cost of one categorical draw.

(ii) **The best factorized proposal is the target’s marginals.** Among factorized distributions $\tilde{p}(\mathbf{x}_0) = \prod_l \tilde{p}_l(\mathbf{x}_0^l)$, the one minimizing $\text{KL}(q \parallel \tilde{p})$ is obtained by matching marginals. Writing it out,

$$\text{KL}(q \parallel \prod_l \tilde{p}_l) = -H(q) - \sum_l \sum_k q_l(k) \log \tilde{p}_l(k), \quad (5)$$

which is maximized term by term at $\tilde{p}_l = q_l$, the per-position marginal of the target. So the object we want is

$$q_l(k) = \sum_{\mathbf{x}_0: \mathbf{x}_0^l = k} q(\mathbf{x}_0 \mid \mathbf{x}_t) \propto p_\theta(\mathbf{x}_0^l = k \mid \mathbf{x}_t) \cdot \underbrace{\mathbb{E}_{\mathbf{x}_0^{-l} \sim p_\theta(\cdot \mid \mathbf{x}_t)} \left[e^{\lambda r(\mathbf{x}_0^{-l}, k)} \right]}_{=: g_l(k)}, \quad (6)$$

where the factorization of p_θ was used to drop the conditioning inside the expectation. Equation (6) is the derivation that was missing: **the exact per-position multiplier is not an arbitrary $\exp(\cdot)$ — it is the expected exponentiated reward obtained by pinning position l to token k and letting the model fill in the rest.**

$g_l(k)$ is intractable: evaluating it by Monte Carlo needs $L \cdot V$ reward calls per sample, and r is a black box, so there is no closed form. Any usable method must replace g_l with an estimate.

4.2 The classifier is an amortized estimate of g_l

Now the reason D-CBG’s score belongs here. Take the tilt in its class-conditional form, $q(\mathbf{x}_0) \propto p_\theta(\mathbf{x}_0) p(c \mid \mathbf{x}_0)^\gamma$, which is what D-CBG’s classifier is trained for ($c =$ “QED above the 90th percentile”). Substituting into (6) with $\gamma = 1$,

$$g_l(k) = \mathbb{E}_{\mathbf{x}_0^{-l} \sim p_\theta(\cdot \mid \mathbf{x}_t)} \left[p(c \mid \mathbf{x}_0^{-l}, k) \right] = \Pr(c \mid \mathbf{x}_t, \mathbf{x}_0^l = k). \quad (7)$$

That is exactly the quantity a noisy classifier is trained to produce: the probability of the target class given the current latent, with one position pinned. D-CBG’s score $s_l(k) = \log p_\phi(c \mid \mathbf{x}_t^{l \leftarrow k}, t)$ is a *learned amortization of (7)* — it replaces an $L \cdot V$ -fold Monte Carlo integral over the model’s own completions with one network evaluation. That, and not analogy with (2), is why multiplying the denoiser by $e^{\gamma s_l(k)}$ is the natural move: it is the plug-in estimator of the optimal factorized proposal (6).

Accordingly, define

$$\tilde{p}_\theta(\mathbf{x}_0^l = k \mid \mathbf{x}_t) = \frac{1}{Z_l} p_\theta(\mathbf{x}_0^l = k \mid \mathbf{x}_t) \exp(\gamma s_l(k)), \quad Z_l = \sum_{k'} p_\theta(\mathbf{x}_0^l = k' \mid \mathbf{x}_t) e^{\gamma s_l(k')}. \quad (8)$$

It is normalizable in closed form, factorized (so drawing N candidates still costs one categorical sample), and costs one classifier evaluation per step shared across all N candidates.

Where the plug-in is inexact, and why that is tolerable. Three gaps separate $e^{\gamma s_l(k)}$ from $g_l(k)$:

1. **Substitution versus conditioning.** p_ϕ is evaluated at $\mathbf{x}_t^{l \leftarrow k}$ — token k written into the *noisy* latent — whereas (7) conditions on $\mathbf{x}_0^l = k$ in the *clean* sequence. These coincide only in the low-noise limit.
2. **Binary class versus continuous reward.** p_ϕ predicts a thresholded indicator, while our target uses $e^{\lambda r}$ with r continuous. The exponent γ is a free parameter that partially absorbs this mismatch, which is another reason it must be retuned rather than copied from D-CBG.

3. Estimation error in p_ϕ itself.

By fact (i) none of these threaten correctness: the weights in (12) divide the entire factor back out, so a wrong g_l estimate costs effective sample size and nothing else. This is the whole reason the combination is worth doing — it lets us use a crude, cheap, possibly misspecified estimate of an intractable quantity without inheriting its bias.

Even a perfect g_l leaves work for the weights. Note that matching the marginals exactly would *not* make the weights uniform. The target q does not factorize — r is a whole-molecule functional, and that coupling is precisely the part a per-position proposal cannot represent. What remains in the weights after an ideal proposal is exactly the position-coupling information, which is the part only the exact reward can supply. The division of labour is therefore not merely convenient but structural: the classifier can in principle supply all of the factorizable signal and none of the rest.

A classifier-free alternative. Equation (6) also suggests a variant needing no classifier at all: estimate $g_l(k)$ from the candidates we already draw, by taking the $M \ll N$ highest-reward candidates and re-scoring single-position substitutions on them. This costs $M \cdot L \cdot V$ reward calls per step, which the optimized reward path makes borderline affordable, and it removes the t -mismatch of item 1 entirely because the reward is evaluated on clean sequences. Recorded as an option; the classifier route is far cheaper.

4.3 The importance weights

Sampling from $\tilde{p}_\theta$ instead of p_θ requires the weights to carry the likelihood ratio. For the target $q(\mathbf{x}_0) \propto p_\theta(\mathbf{x}_0) \exp(\lambda r(\mathbf{x}_0))$,

$$w_n \propto \exp\left(\lambda r(\mathbf{x}_0^{(n)})\right) \frac{p_\theta(\mathbf{x}_0^{(n)} \mid \mathbf{x}_t)}{\tilde{p}_\theta(\mathbf{x}_0^{(n)} \mid \mathbf{x}_t)} \quad (9)$$

$$= \exp\left(\lambda r(\mathbf{x}_0^{(n)})\right) \prod_{l=1}^L Z_l \exp\left(-\gamma s_l(\mathbf{x}_0^{(n),l})\right) \quad (10)$$

$$\propto \exp\left(\lambda r(\mathbf{x}_0^{(n)}) - \gamma \sum_{l=1}^L s_l(\mathbf{x}_0^{(n),l})\right), \quad (11)$$

where the last step drops $\prod_l Z_l$ because it does not depend on n and therefore cancels in the self-normalization. So the whole combination reduces to a single extra term in the log-weight:

$$\boxed{\log w_n = \underbrace{\lambda r(\mathbf{x}_0^{(n)})}_{\text{exact reward}} - \underbrace{\gamma \sum_l \log p_\phi\left(c \mid \mathbf{x}_t^{l \leftarrow \mathbf{x}_0^{(n),l}}\right)}_{\text{undo the proposal tilt}} + \text{const},} \quad (12)$$

followed by a softmax over n . Nothing else in the algorithm changes: $\bar{\mathbf{x}}_0$, the posterior substitution, and the time window all stay as they are.

4.4 Why this is the right object

Read (8)–(12) as a statement about roles. Section 4.1 established that the best factorized proposal is the target’s per-position marginals (6), and that D-CBG’s classifier is an amortized estimate of exactly that object. In this framing,

- **D-CBG alone** uses the surrogate and never corrects it, so it is biased with respect to the true tilt by however wrong p_ϕ is;
- **ours alone** uses the exact tilt but places its atoms with the untilted denoiser, so it is consistent but support-limited;
- **combined**, the surrogate places the atoms where the reward is plausibly high and the exact reward corrects whatever the surrogate got wrong.

Crucially the correction makes the combination *robust to a bad classifier*: p_ϕ enters only through the proposal, and (12) divides its influence back out. A poor classifier costs effective sample size, never correctness. That is a strictly better contract than D-CBG’s, where classifier error is indistinguishable from signal.

Limiting cases. Both reduce correctly.

- $\gamma \rightarrow 0$: the proposal becomes p_θ and the correction term vanishes, recovering our current method exactly.
- $\lambda \rightarrow 0$: the weights become $w_n \propto \prod_l e^{-\gamma s_l(\mathbf{x}_0^{(n),l})}$, which exactly undoes the proposal tilt, so $\bar{\mathbf{x}}_0 \rightarrow \mathbf{x}_\theta$ and sampling reduces to the unguided base model. This is a useful implementation check: with $\lambda = 0$ the combined sampler must reproduce unguided statistics no matter what γ is.

4.5 A bias/variance knob

Equation (12) fully removes the classifier tilt. Removing it only partially,

$$\log w_n = \lambda r(\mathbf{x}_0^{(n)}) - \beta \gamma \sum_l s_l(\mathbf{x}_0^{(n),l}), \quad \beta \in [0, 1], \quad (13)$$

interpolates between two *different targets* rather than merely trading variance:

- $\beta = 1$ targets $q(\mathbf{x}_0) \propto p_\theta(\mathbf{x}_0) e^{\lambda r(\mathbf{x}_0)}$ — the reward-tilted distribution, classifier used purely as a proposal.
- $\beta = 0$ targets the product of experts $q(\mathbf{x}_0) \propto p_\theta(\mathbf{x}_0) e^{\lambda r(\mathbf{x}_0)} \prod_l p_\phi(c \mid \mathbf{x}_t^{l \leftarrow \mathbf{x}_0^l})^\gamma$ — the classifier’s opinion is kept as an additional factor in the objective.

Neither is obviously preferable: $\beta = 0$ is not a bug if one is willing to define the target that way, and it will have higher ESS because the weights no longer fight the proposal. $\beta = 1$ is the principled choice if the reward is the definition of what we want.

4.6 Scale warning

The two terms in (12) are not on comparable scales, and reusing D-CBG’s tuned γ directly will not work. In D-CBG, $\gamma s_l(k)$ competes against a *per-position* log-posterior. In (12) it appears as $\gamma \sum_l s_l$, a sum over all $L = 32$ positions, competing against the scalar λr . Expect the useful γ here to be smaller than D-CBG’s by roughly a factor of L , and expect the correction term to dominate the reward unless γ is retuned from scratch. Monitoring ESS during the sweep is the practical guard: a collapse to $\text{ESS} \approx 1$ means the correction term is swamping the reward.

5 Algorithm

Algorithm 1 One guided denoising step (combined)

- 1: $\log \mathbf{x}_\theta \leftarrow \text{denoiser}(\mathbf{x}_t, \sigma_t)$ ▷ 1 network forward
 - 2: $s_l(k) \leftarrow \log p_\phi(c \mid \mathbf{x}_t^{l \leftarrow k}, t)$ for all l, k ▷ 1 forward + backward (approx.), or LV forwards (exact)
 - 3: $\tilde{p}_\theta(\mathbf{x}_0^l = k \mid \mathbf{x}_t) \propto \mathbf{x}_\theta[l, k] e^{\gamma s_l(k)}$ ▷ normalize per position
 - 4: $\mathbf{x}_0^{(n)} \sim \tilde{p}_\theta(\cdot \mid \mathbf{x}_t)$, $n = 1 \dots N$ ▷ one categorical draw; free in N
 - 5: $\log \tilde{w}_n \leftarrow \lambda r(\mathbf{x}_0^{(n)}) - \beta \gamma \sum_l s_l(\mathbf{x}_0^{(n), l})$ ▷ N RDKit calls, deduplicated and pooled
 - 6: $w \leftarrow \text{softmax}_n(\log \tilde{w})$; $\bar{\mathbf{x}}_0 \leftarrow \sum_n w_n \text{onehot}(\mathbf{x}_0^{(n)})$
 - 7: $\mathbf{x}_{t-1} \sim q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \bar{\mathbf{x}}_0)$
-

Cost. One network forward plus one classifier forward/backward per step, independent of N ; the reward calls scale with the number of *distinct* candidates. Relative to our current sampler this adds only the classifier pass, so with `use_approx=True` the combined method should cost roughly what D-CBG costs, while retaining our method’s N -scaling. Use the approximate branch: the exact one needs $L \cdot V$ classifier forwards and already exhausted a 3090’s memory at batch size 64.

6 Open issues

1. **How good an estimate of g_l is s_l ?** This is now a well-posed empirical question rather than a vague worry: (7) says the target is $\Pr(c \mid \mathbf{x}_t, \mathbf{x}_0^l = k)$, and it can be estimated directly, if expensively, by Monte Carlo over model completions. Measuring the agreement between $e^{\gamma s_l}$ and that estimate as a function of t would say exactly where the classifier route is worth using. The time-window results predict poor agreement at large t , which argues for switching the classifier tilt on only inside the same window where the reward is informative.
2. **Weight blow-up.** The correction $e^{-\gamma \sum_l s_l}$ is large exactly for candidates the classifier disliked. If the classifier and the reward disagree, a single candidate can dominate. Clipping the log-weights, or $\beta < 1$, are the obvious mitigations.
3. **Absorbing state.** On unmasked positions $\mathbf{x}_{t-1}$ is forced to copy $\mathbf{x}_t$, so both the proposal tilt and the sum in (12) should run over masked positions only; including copied positions adds a constant to every $\log w_n$, which cancels, but only if the same set of positions is used for all n — which it is, since masking depends on $\mathbf{x}_t$ alone.
4. **Does it actually help?** The empirical claim to test is that at matched N the combined method attains higher ESS and higher novel QED than either component. The companion

note’s finding that our method’s gain generalizes to novel molecules while D-CBG’s concentrates on recall suggests the combination should be evaluated on novel QED specifically, not on the all-valid mean.